

# Optimisation in dMod2

Hessian sources, seeding and the cost of a multi-start

Simon Beyer

2026-09-20

## Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                         | <b>2</b>  |
| <b>2</b> | <b>The trust-region core</b>                | <b>3</b>  |
| 2.1      | The subproblem . . . . .                    | 3         |
| 2.2      | The step and the radius . . . . .           | 4         |
| 2.3      | Bounds . . . . .                            | 4         |
| 2.4      | Termination . . . . .                       | 4         |
| <b>3</b> | <b>The source of the model Hessian</b>      | <b>5</b>  |
| 3.1      | BFGS and SR1 . . . . .                      | 5         |
| 3.2      | The handover . . . . .                      | 6         |
| <b>4</b> | <b>Seed and memory of the approximation</b> | <b>7</b>  |
| <b>5</b> | <b>Multi-start on Boehm</b>                 | <b>7</b>  |
| 5.1      | Boehm without the prior . . . . .           | 9         |
| <b>6</b> | <b>Bachmann, 113 parameters</b>             | <b>10</b> |
| <b>7</b> | <b>Discussion</b>                           | <b>12</b> |
| <b>8</b> | <b>Reproducing the numbers</b>              | <b>12</b> |
|          | <b>References</b>                           | <b>13</b> |

# 1 Introduction

Parameter estimation for ordinary differential equation models is a non-convex problem, solved by local optimisation from many starting points. dMod2 [1] provides the trust-region method `trust()` and `mstrust()`, which runs it from many starts. This vignette describes their arguments and the behaviour they produce.

The running example is the STAT5 dimerisation model of Boehm et al. [2], built as in the shipped example `example_Boehm_JProteomeRes2014.R`: nine parameters on a base-10 logarithmic scale, a weak prior on the six dynamic ones, and bounds of  $\pm 5$ . The multi-start comparisons on this model share one set of 100 starting points; the comparison without the prior and the Bachmann model use 200.

```
data(boehm)
mydataL <- as.datalist(boehm)

epo <- "1.25e-7*exp(-Epo_degradation_BaF3*time)"
reactions <- eqnlist() |>
  assignCompartment(nucpApA = "nuc", nucpApB = "nuc", nucpBpB = "nuc",
    volume = "0.45") |>
  addReaction("2*STAT5A", "pApA", paste0("k_phos*", epo, "*STAT5A^2"),
    compartment = "cyt") |>
  addReaction("STAT5A + STAT5B", "pApB", paste0("k_phos*", epo, "*STAT5A*STAT5B"),
    compartment = "cyt") |>
  addReaction("2*STAT5B", "pBpB", paste0("k_phos*", epo, "*STAT5B^2"),
    compartment = "cyt") |>
  addReaction("pApA", "nucpApA", "k_imp_homo*pApA", compartment = "cyt") |>
  addReaction("pApB", "nucpApB", "k_imp_hetero*pApB", compartment = "cyt") |>
  addReaction("pBpB", "nucpBpB", "k_imp_homo*pBpB", compartment = "cyt") |>
  addReaction("nucpApA", "2*STAT5A", "k_exp_homo*nucpApA", compartment = "nuc") |>
  addReaction("nucpApB", "STAT5A + STAT5B", "k_exp_hetero*nucpApB",
    compartment = "nuc") |>
  addReaction("nucpBpB", "2*STAT5B", "k_exp_homo*nucpBpB", compartment = "nuc") |>
  setCompartmentVolume(cyt = "1.4")

# The optimiser can only resolve what the integrator delivers, so the tolerances
# are set explicitly rather than left at their defaults.
myOptionsODE <- list(atol = 1e-8, rtol = 1e-6, maxattempts = 100L, maxsteps = 1e6)
model <- odemodel(reactions, modelname = "boehm_ode", compile = FALSE,
  outdir = .outdir)
x <- Xs(model, optionsOde = myOptionsODE, optionsSens = myOptionsODE)

observables <- eqnvec(
  pSTAT5A_rel = paste0("(100*pApB + 200*pApA*specC17)/",
    "(pApB + STAT5A*specC17 + 2*pApA*specC17)"),
  pSTAT5B_rel = paste0("-(100*pApB - 200*pBpB*(specC17 - 1))/",
    "((STAT5B*(specC17 - 1) - pApB) + 2*pBpB*(specC17 - 1))"),
  rSTAT5A_rel = paste0("(100*pApB + 100*STAT5A*specC17 + 200*pApA*specC17)/",
    "(2*pApB + STAT5A*specC17 + 2*pApA*specC17",
    " - STAT5B*(specC17 - 1) - 2*pBpB*(specC17 - 1))"))
errorModels <- eqnvec(pSTAT5A_rel = "sd_pSTAT5A_rel",
  pSTAT5B_rel = "sd_pSTAT5B_rel",
  rSTAT5A_rel = "sd_rSTAT5A_rel")

g <- Y(observables, x, modelname = "boehm_obs", attach.input = FALSE,
  compile = FALSE, outdir = .outdir)
e <- Y(errorModels, g, modelname = "boehm_err", attach.input = FALSE,
  compile = FALSE, outdir = .outdir)

innerpars <- getParameters(model, g, e)
estimated <- c("Epo_degradation_BaF3", "k_exp_hetero", "k_exp_homo",
  "k_imp_hetero", "k_imp_homo", "k_phos",
```

```

      "sd_pSTAT5A_rel", "sd_pSTAT5B_rel", "sd_rSTAT5A_rel")
p <- eqnvec() |>
  define("x-x", x = innerpars) |>
  define("x-0", x = c("pApA", "pApB", "pBpB", "nucpApA", "nucpApB", "nucpBpB")) |>
  insert("STAT5A ~ 207.6*ratio") |>
  insert("STAT5B ~ 207.6 - 207.6*ratio") |>
  insert("ratio ~ 0.693") |>
  insert("specC17 ~ 0.107") |>
  insert("x-exp10(x)", x = estimated) |>
  P(modelname = "boehm_trafo", condition = "Boehm2014", compile = FALSE,
    outdir = .outdir)

compile(g, x, p, e, output = "boehm_vig", cores = 2)

prd      <- g*x*p
outerpars <- getParameters(prd)
pouter   <- structure(rep(-1, length(outerpars)), names = outerpars)
dyn       <- setdiff(outerpars, grep("^sd_", outerpars, value = TRUE))
obj       <- normL2(mydataL, prd, e) + constraintL2(pouter[dyn], sigma = 4)

parlower <- structure(rep(-5, length(outerpars)), names = outerpars)
parupper <- structure(rep( 5, length(outerpars)), names = outerpars)

length(pouter)
#> [1] 9

```

## 2 The trust-region core

`trust()` minimises a smooth objective by solving in each iteration the quadratic subproblem

$$\min_{\|p\| \leq r} g^\top p + \frac{1}{2} p^\top H p,$$

exactly, through a full eigendecomposition of  $H$  [3]. Box constraints are handled by the interior scheme of Coleman and Li [4], which keeps every iterate strictly inside the box; the `atBound` component of the result reports which parameters a bound is holding.

The subproblem is solved exactly rather than by truncated conjugate gradients because the quasi-Newton updates below can make  $H$  indefinite: the Moré-Sorensen solver handles that case directly, whereas a Steihaug-type method can only stop at the trust-region boundary.

### 2.1 The subproblem

Moré and Sorensen [3] characterise the solution by a scalar. With the eigendecomposition  $H = Q\Lambda Q^\top$  and  $\gamma = Q^\top g$ , a step  $p(\lambda) = -Q(\Lambda + \lambda I)^{-1}\gamma$  solves it exactly when  $\lambda \geq \max(0, -\lambda_{\min})$  and either  $\lambda = 0$  with  $\|p\| \leq r$ , or  $\lambda > 0$  with  $\|p\| = r$ . The second case is a root of the secular equation

$$\phi(\lambda) = \frac{1}{r} - \frac{1}{\|p(\lambda)\|} = \frac{1}{r} - \left( \sum_i \frac{\gamma_i^2}{(\lambda_i + \lambda)^2} \right)^{-1/2},$$

which is smooth and almost linear on  $\lambda > -\lambda_{\min}$ , so Newton's method on  $\phi$  converges in a few iterations. Given the one eigendecomposition per iteration, each Newton step in  $\lambda$  costs  $O(n_\theta)$ .

A positive definite model never reaches the hard case: when  $\gamma_i = 0$  for every eigenvector of  $\lambda_{\min}$ , no  $\lambda$  drives  $\|p\|$  out to  $r$ , and the solution is  $p = p(-\lambda_{\min}) + \tau q_{\min}$  with  $\tau$  chosen so that  $\|p\| = r$ . An indefinite SR1 model produces this case.

## 2.2 The step and the radius

Each iteration compares the decrease predicted by the model with the actual decrease,

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{-m_k(p_k)}, \quad m_k(p) = g_k^\top p + \frac{1}{2} p^\top H_k p,$$

and updates the radius from that ratio, with  $r_{\max} = \mathbf{rmax}$ :

$$r_{k+1} = \begin{cases} \min(r_k/4, \|p_k\|/4), & \rho_k < 1/4, \\ \min(2r_k, r_{\max}), & \rho_k > 3/4 \text{ and } \|p_k\| \geq 0.9 r_k, \\ r_k, & \text{otherwise.} \end{cases}$$

A step with  $\rho_k < 1/4$  is rejected, and a step much shorter than the radius shrinks it to the step's own scale. The radius grows only when the step reaches the boundary, since a shorter step was limited by the model, not by the region.

Under `stepControl$nonmonotone` the actual decrease is measured from a weighted average of past values [5],

$$Q_{k+1} = \eta Q_k + 1, \quad C_{k+1} = \frac{\eta Q_k C_k + f(x_{k+1})}{Q_{k+1}},$$

and  $\rho_k = (C_k - f(x_k + p_k))/(-m_k(p_k))$ . At  $\eta = 0$ ,  $C_k = f(x_k)$  and the rule above is recovered. A larger  $\eta$  lets the iterate leave a narrow valley by accepting steps that raise the objective, so the result reports the best iterate visited, not the last. The termination tests still read the true objective: only acceptance is relaxed.

## 2.3 Bounds

Coleman and Li [4] turn a bound into a change of metric. With  $v_i$  the signed distance from  $x_i$  to whichever bound the gradient pushes it towards,  $D = \text{diag}(|v|^{1/2})$ , and  $J^v$  the diagonal of  $\partial|v|/\partial x$ , the subproblem is solved in the scaled variables

$$\hat{g} = Dg, \quad \hat{B} = DHD + \text{diag}(g)J^v,$$

so the model steepens towards a bound and the step turns away from it. The first-order optimality measure that `gtol` reads,  $\max_i |v_i g_i|$ , vanishes both at an interior stationary point and at a bound the gradient pushes into. A step that would still leave the box is truncated to a fraction `theta.max` of the distance, reflected off the blocking faces, or replaced by the scaled steepest-descent step, whichever the model rates best.

## 2.4 Termination

Six tests can end a run; the `stopReason` component names the one that fired. Which of them apply depends on the Hessian source.

| <code>stopReason</code> | criterion                                                         | evaluated on           |
|-------------------------|-------------------------------------------------------------------|------------------------|
| <code>gradient</code>   | $\max  v \cdot g  \leq \mathbf{gtol}$                             | the current iterate    |
| <code>fvalue</code>     | $ f - f_{\text{try}}  < \mathbf{ftol}$                            | the last accepted step |
| <code>preddiff</code>   | predicted decrease $< \mathbf{mtol}$                              | the last accepted step |
| <code>step</code>       | step norm $< \mathbf{xtol}$                                       | the last accepted step |
| <code>stagnation</code> | five rejected steps leaving $f$ flat                              | recent steps           |
| <code>radius</code>     | radius $< \mathbf{rmin}$ ; reports <code>converged = FALSE</code> | the radius             |

Only **gradient** checks first-order optimality. **fvalue**, **preddiff** and **step** read the prediction of the quadratic model, so a Hessian that misrepresents the local curvature can trip them at a point that is not stationary. This explains a fit reporting **converged** = **TRUE** at an implausibly high objective value; the handover below exploits it.

While the model is the objective's own  $J^T J$ , these three tests apply. During a quasi-Newton phase the model is an approximation still being built, and its prediction describes the approximation rather than the iterate. The three are then suppressed; the run ends on **gradient**, on **stagnation**, which counts rejected steps instead of reading the model, or on **radius**. The gradient norm is the termination criterion usually recommended for quasi-Newton methods [6].

### 3 The source of the model Hessian

**hessianMethod** selects the source of  $H$ . The subproblem solver, the scaling and the bound handling are the same in all four cases.

| value   | Hessian                                          | required from the objective    |
|---------|--------------------------------------------------|--------------------------------|
| "gn"    | the objective's $J^T J$ , rebuilt each iteration | value, gradient, Hessian       |
| "bfgs"  | a Powell-damped BFGS update [7]                  | value, gradient                |
| "sr1"   | a symmetric rank-one update, possibly indefinite | value, gradient                |
| "exact" | the objective's exact Hessian, a Newton run      | value, gradient, exact Hessian |

**hessianFallback** names a second source and **fallbackLimit** how often the two may swap. A run is a method and optionally a fallback; no pair has a name of its own.

For a sum of squares the Gauss-Newton matrix  $J^T J$  is an adequate curvature model wherever the residuals are small, and **dMod2** obtains it at no extra cost: the Jacobian that yields the gradient also yields the Hessian, so a gradient and a full derivative evaluation cost the same. **"gn"** is therefore the default.

The Gauss-Newton matrix omits the term of residuals times their second derivatives. Near the optimum the quadratic model therefore stops improving, and **ftol** or **mtol** fires short of the minimum. A quasi-Newton approximation infers curvature from the steps taken and carries no such bias.

#### 3.1 BFGS and SR1

Both updates use the secant pair

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k),$$

and require the new model to reproduce it,  $B_{k+1}s_k = y_k$ .

**BFGS** is the rank-two update that stays positive definite, which requires  $s^T y > 0$ . Away from a minimum this curvature condition can fail, so the pair is damped before use [7]: with

$$\theta = \begin{cases} 1, & s^T y \geq 0.2 s^T B s, \\ \frac{0.8 s^T B s}{s^T B s - s^T y}, & \text{otherwise,} \end{cases} \quad \tilde{y} = \theta y + (1 - \theta) B s,$$

the update is

$$B_{k+1} = B_k - \frac{B_k s s^\top B_k}{s^\top B_k s} + \frac{\tilde{y} \tilde{y}^\top}{s^\top \tilde{y}}.$$

The damped pair satisfies  $s^\top \tilde{y} \geq 0.2 s^\top B s > 0$ , so  $B$  stays positive definite. **qnCautious** adds the test of Li and Fukushima [8]: a pair counts only when  $s^\top y > c \|s\| \|y\|$  with  $c = \mathbf{qnCautious}$ , which discards pairs whose curvature is numerical noise.

**SR1** is the rank-one update that satisfies the secant equation without requiring definiteness. With  $w = y - B_k s$ ,

$$B_{k+1} = B_k + \frac{w w^\top}{w^\top s},$$

the only symmetric rank-one correction that does so. Since  $w^\top s$  can be arbitrarily small, the update is skipped when  $|w^\top s| \leq 10^{-8} \|s\| \|w\|$ . SR1 converges to the true Hessian, indefinite regions included. That is of no use to a line search but suits a trust region: the negative curvature it reports is real, and the Moré-Sorensen solver can step along it.

For SR1 **qnRejected** is on: a rejected trial point still supplies a valid  $(s, y)$ , because the curvature along a step does not depend on whether the step is accepted, and the trial gradient is already computed [6]. BFGS cannot use such a pair, since its positive definiteness argument rests on the step being accepted.

### 3.2 The handover

A fallback run stays on "gn" until **fvalue**, **preddiff**, **step** or **stagnation** would terminate it, then switches to the fallback source, seeded with the last accepted  $J^\top J$ . **gradient** remains terminal: it is an optimality test, not a symptom of a stalled model. The construction follows Fides of Fröhlich and Sorger [9], which switches when the trust radius stops updating. The **qnEval** component counts the objective evaluations after the switch.

```
set.seed(1L)
start <- as.parvec(msParframe(pouter, n = 1, sd = 3))

single <- .fit("single", list(
  gn      = trust(obj, start, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper),
  fallback = trust(obj, start, rinit = 0.1, rmax = 10, iterlim = 5000,
    parlower = parlower, parupper = parupper,
    hessianFallback = "bfgs")))

data.frame(method = names(single),
  value = vapply(single, `[`, 0.0, "value"),
  stop = vapply(single, `[`, "", "stopReason"),
  neval = vapply(single, `[`, 0L, "neval"),
  qnEval = vapply(single, `[`, 0L, "qnEval"),
  row.names = NULL)

#>   method   value      stop neval qnEval
#> 1    gn 297.3323    fvalue   105      0
#> 2 hybrid 297.3260 stagnation  143     38
```

Over the shared starting points of the next section a fallback run uses a few per cent more evaluations than "gn". The **grad** column there counts the starts ending at a stationary point, which a fallback reaches and "gn" does not.

## 4 Seed and memory of the approximation

Three arguments shape the approximation. They apply when a run begins in a quasi-Newton phase, that is for "bfgs" and "sr1"; "gn" ignores them, and a fallback seeds its handover from the Gauss-Newton matrix irrespective of `hessianInit`.

`hessianInit` selects the initial matrix: the default "gn" is the objective's Hessian at `parinit`, "identity" the identity. With "identity", `trust()` never requests a Hessian, so an objective that returns only gradients suffices.

In a multi-start a genuine Hessian is not the better seed, and the multi-start below seeds "bfgs" and "sr1" with the identity; the later comparisons also run "sr1" from a Gauss-Newton seed. At a starting point three decades from the optimum,  $J^T J$  is large and severely ill-conditioned. A BFGS update is of rank two and preserves positive definiteness, so it cannot reduce such a seed along the directions where it is wrong by orders of magnitude, and the wrong curvature keeps dominating the approximation; on this model those runs exhaust the iteration cap without reaching a termination test. "sr1" is not affected: its rank-one update has no sign constraint and overwrites bad curvature outright, and the Moré-Sorensen subproblem solves the indefinite approximations that follow. `inst/benchmarks` compares both seeds.

`qnMemory` limits the curvature carried forward. The default 0, used in the rest of this vignette, accumulates every update onto the seed. A positive value rebuilds the approximation in each iteration from  $\gamma I$  and the most recent `qnMemory` pairs, where  $\gamma$  is the scaling  $y^T y / s^T y$  of Shanno and Phua [10] from the newest pair. This is the curvature information of L-BFGS [6], assembled into the explicit matrix the subproblem requires. It saves no memory; the seed and every pair older than the last `qnMemory` are forgotten.

`qnCautious`, the cautious-update test above, changes neither the outcome nor the cost on this model; `qnSkipped` counts the pairs it rejects.

```
seeds <- .fit("seeds", lapply(list(
  list(hessianMethod = "bfgs"),
  list(hessianMethod = "bfgs", qnControl = list(hessianInit = "identity")),
  list(hessianMethod = "bfgs",
    qnControl = list(hessianInit = "identity", qnMemory = 10L))),
  function(a) do.call(trust, c(list(obj, start, rinit = 0.1, rmax = 10,
    iterlim = 5000, parlower = parlower,
    parupper = parupper), a))))

data.frame(seed = c("gn", "identity", "identity"),
  memory = c(0L, 0L, 10L),
  value = vapply(seeds, `[`, 0.0, "value"),
  stop = vapply(seeds, `[`, "", "stopReason"),
  neval = vapply(seeds, `[`, 0L, "neval"))

#>      seed memory      value      stop neval
#> 1      gn      0 297.3260 stagnation    107
#> 2 identity      0 279.6607 stagnation    139
#> 3 identity     10 499.4909 stagnation     84
```

## 5 Multi-start on Boehm

The starting points are drawn once with `msParframe()` and passed to `mstrust()` as `center`, so a comparison between methods differs only in the method.

```
set.seed(20260905)
starts <- msParframe(pouter, n = 100, sd = 3)

fitAll <- function(...)
  as.parframe(mstrust(obj, center = starts, fits = nrow(starts),
    cores = .cores, rinit = 0.1, rmax = 10, iterlim = 5000,
```

```

        parlower = parlower, parupper = parupper, ...))

source(system.file("benchmarks", "hessianSourceSettings.R", package = "dMod2"))
settings <- hessianSourceSettings[c("gn", "gn_bfgs", "gn_sr1", "bfgs_id", "sr1_id")]

runs <- .fit("multistart", lapply(settings, function(a) do.call(fitAll, a)))
names(runs) <- hessianSourceLabels[names(settings)]

```

A start is a hit when it ends within 0.1 of the best value over all runs.

```

best <- min(vapply(runs, function(f) min(f$value), 0.0))

summary <- function(f) {
  hits <- sum(f$value <= best + 0.1)
  data.frame(conv      = sum(f$converged),
             grad      = sum(f$stopReason == "gradient"),
             hits      = hits,
             neval     = sum(f$neval),
             medEval   = stats::median(f$neval),
             # The cost of one start that reaches the optimum, as reported by
             # the Bachmann run.
             evalPerHit = if (hits > 0) round(sum(f$neval) / hits) else NA_real_)
}

cbind(source = names(runs), do.call(rbind, lapply(runs, summary)),
      row.names = NULL)

#>      source conv grad hits neval medEval evalPerHit
#> 1      gn  100   0    6 35536   219.0     5923
#> 2 gn -> bfgs 100  13    6 38517   244.5     6420
#> 3 gn -> sr1  100  28    6 37290   240.0     6215
#> 4  bfgs, id 100  30   13 11162   120.5      859
#> 5  sr1, id  100  19   34 19726   207.5     580

```

A waterfall plot sorts the final objective values in ascending order. A flat leading plateau indicates repeated arrival at the same optimum, the height of the first step how well that optimum is separated from the next. `plotValues()` draws one collection, shaded by iteration count; the figure below ranks the five sources separately and colours them by source. A circle marks a converged fit, a triangle one that stopped on any other test.

```

# One panel for several fit collections, each ranked on its own. Drawn back to
# front, so the first one listed is not buried, and without the palette's
# pale yellow.
waterfall <- function(frames, cutoff = Inf) {
  ranked <- do.call(rbind, lapply(rev(names(frames)), function(set) {
    fits <- subset(frames[[set]], value < cutoff)
    fits <- fits[order(fits$value), ]
    data.frame(index = seq_len(nrow(fits)), value = fits$value,
               converged = as.character(fits$converged), set = set)
  }))
  ranked$set <- factor(ranked$set, levels = names(frames))
  colours <- stats::setNames(dMod_palette(8, "okabe")[-5][seq_along(frames)],
                             names(frames))

  ggplot(ranked, aes(index, value, colour = set)) +
    geom_line(aes(group = set), linewidth = 0.3) +
    geom_point(aes(shape = converged), size = 1.3) +
    scale_shape_manual(values = c(`TRUE` = 16, `FALSE` = 17),
                       limits = c("TRUE", "FALSE"), labels = c("yes", "no")) +
    scale_colour_manual(values = colours) +

```

```

labs(x = "index", y = "value", colour = NULL) +
theme_dMod() +
theme(legend.position = "bottom", legend.box = "vertical")
}

waterfall(runs, cutoff = 600)

```

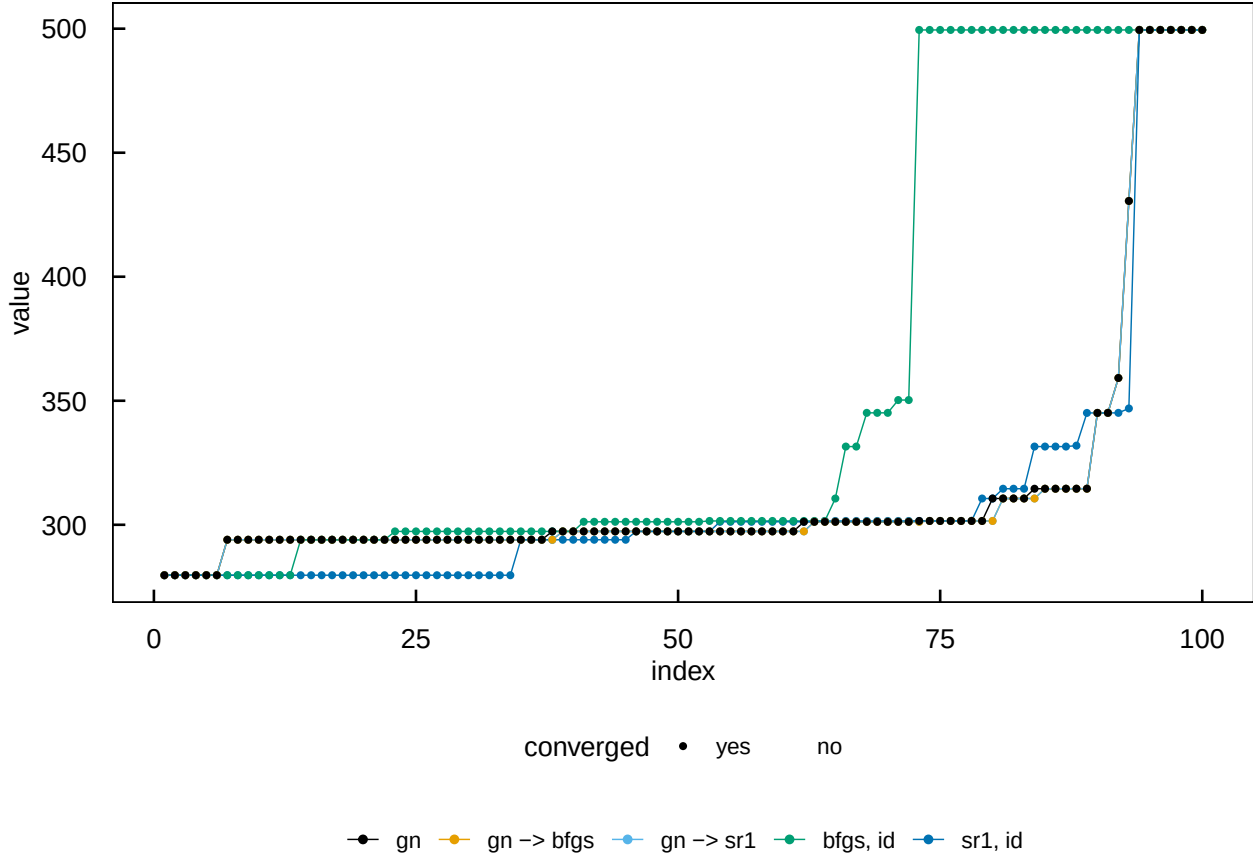


Figure 1: Waterfall plots of the five Hessian sources over the same 100 starting points, each ranked separately. Values above 600 are omitted.

The curves fall into two shapes. Gauss-Newton and the two fallback runs track each other over the whole range, since a fallback is Gauss-Newton until a soft stop fires. They differ in the leading plateau: under Gauss-Newton it is not flat, because the model stops improving before the minimum, whereas the quasi-Newton phase carries those starts onto a common value. The identity-seeded runs form the second shape, with a longer leading plateau: they reach the global optimum from more starting points, and more of the remaining starts end high. Under "bfgs" this worsens the typical value, under "sr1" it does not.

By `stopReason`, Gauss-Newton ends on the model tests, which are valid for it, and the quasi-Newton runs, where those tests are suppressed, end on `stagnation` or `gradient`.

## 5.1 Boehm without the prior

The prior keeps starts away from the bounds and shapes the comparison. Two hundred starts were therefore also run on the likelihood alone, with the bounds widened to  $\pm 8$ . The published optimum is  $-2 \log L = 276.44$  [11]; a hit ends within 0.1 of it.

| source          | best    | reaching the optimum | median evaluations | evaluations per optimum |
|-----------------|---------|----------------------|--------------------|-------------------------|
| "gn"            | 276.396 | 28 of 200            | 98                 | 4236                    |
| "gn" to "sr1"   | 276.396 | 28 of 200            | 133                | 4452                    |
| "sr1", identity | 276.396 | 32 of 200            | 219                | 4190                    |
| "sr1", gn seed  | 276.396 | <b>40 of 200</b>     | 226                | 2440                    |
| "sr1" to "gn"   | 276.396 | 35 of 200            | 172                | <b>2411</b>             |

```
noprior <- readRDS("boehm_noprior.rds")
waterfall(split(noprior[c("value", "converged")], noprior$source), cutoff = 600)
```

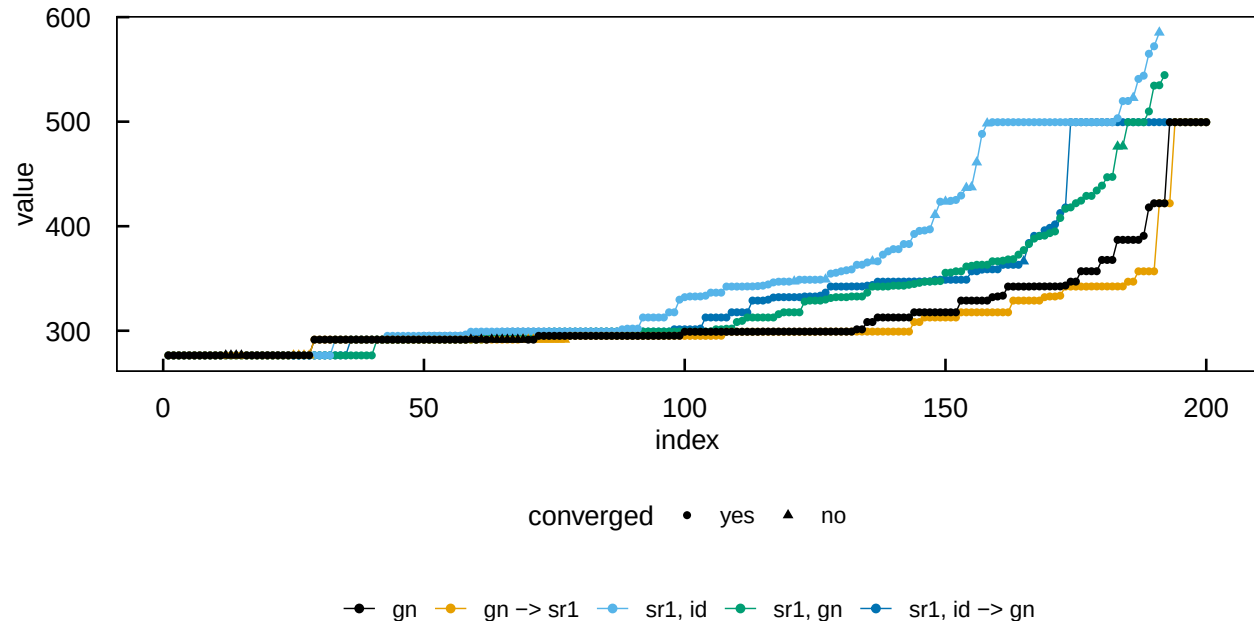


Figure 2: Waterfall plots of five Hessian sources on the Boehm likelihood without a prior, over the same 200 starting points, each ranked separately. Values above 600 are omitted.

Without the prior every source still reaches the published optimum, and the quasi-Newton runs lose most of their lead: identity-seeded "sr1" reaches it from 32 of 200 starts against 34 of 100 with the prior, at seven times the evaluations per optimum, while "gn" gains, 28 of 200 against 6 of 100. "sr1" from a Gauss-Newton seed reaches the optimum from the most starts, and the reversed handover at the fewest evaluations per optimum, with a fifth to a quarter fewer evaluations per start than either "sr1" run.

The reversed handover also changes how runs end. "sr1" alone stagnates on 156 of 200 starts; handing back to "gn" turns that into 159 ends on `fvalue` and 11 on stagnation, and cuts runs exhausting the iteration cap from 13 to 4. The forward handover matches "gn" on every count, as on Bachmann.

## 6 Bachmann, 113 parameters

A dense quasi-Newton update needs of the order of  $n_\theta$  accepted steps before it carries curvature along every direction, so its advantage should narrow as the problem grows. The JAK2-STAT5 model of Bachmann et al. [12] tests this with 113 estimated parameters, 25 states, 20 observables and 36 conditions. The fits ran on a SLURM cluster through `dMod2::distributedComputing()`, one job per source, each an array of 50 tasks of four fits.

A multi-start over 113 parameters needs a weak prior, or starts run into the bounds instead of converging.

A single prior centre for all parameters is wrong where the published values span twelve decades: at  $\sigma = 4$  it adds 27 to the objective at the published optimum and moves the optimum. For every parameter whose published value lies more than a decade from the default centre, the centre is therefore moved to the order of magnitude of that value, so it lies within one decade everywhere and the same  $\sigma = 4$  adds 1.04. The optimum stays at the published parameters, where the objective, prior included, is  $-479.26$ ; a hit ends within 0.1 of it. Starts are drawn around that centre and redrawn where they leave the box.

| source          | best     | reaching the optimum | median evaluations | evaluations per optimum |
|-----------------|----------|----------------------|--------------------|-------------------------|
| "gn"            | -479.264 | 83 of 200            | 141                | <b>638</b>              |
| "gn" to "sr1"   | -479.264 | 83 of 200            | 156                | 678                     |
| "sr1", identity | -479.264 | 79 of 200            | 500                | 1292                    |
| "sr1", gn seed  | -479.264 | 65 of 200            | 592                | 1926                    |
| "sr1" to "gn"   | -479.264 | 79 of 200            | 490                | 1275                    |

```
# Cluster fits, so they sit beside the document as data rather than in the
# refit cache: a --refit cannot recompute them.
bachmann <- readRDS("bachmann.rds")
waterfall(split(bachmann[c("value", "converged")], bachmann$source), cutoff = -400)
```

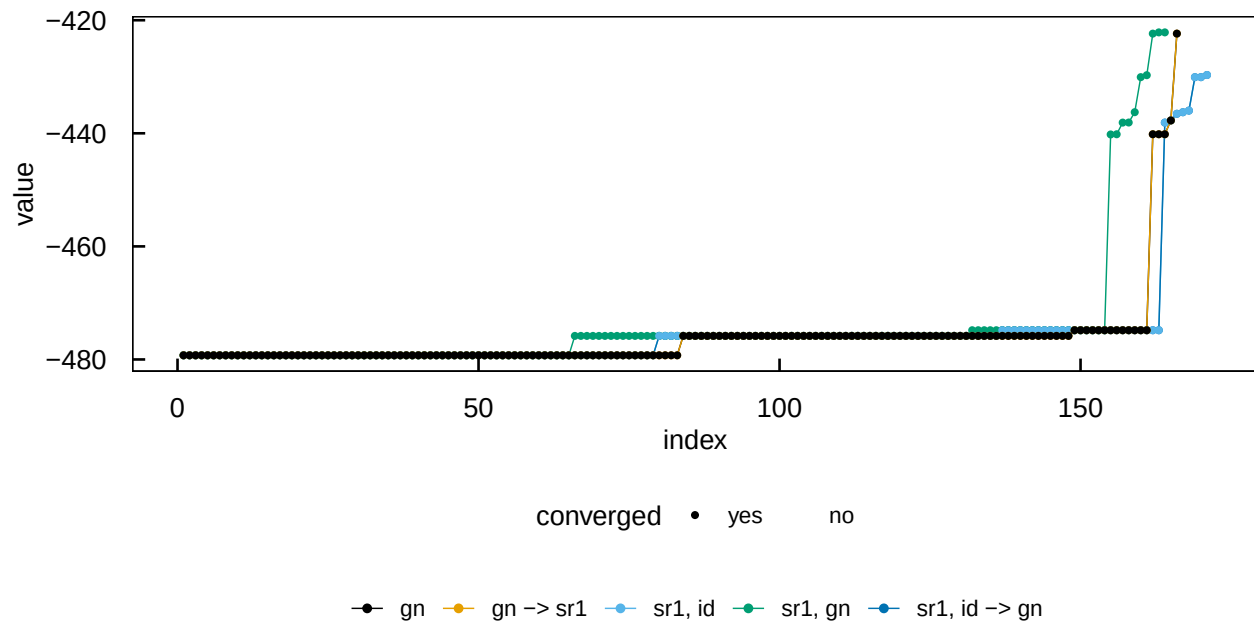


Figure 3: Waterfall plots of five Hessian sources on Bachmann over the same 200 starting points, each ranked separately. Starts ending above  $-400$  are omitted.

Every source reaches the same best value, and the counts at 0.1 and at 1.0 from the optimum are identical: a start either finds the published minimum or ends far from it. No source finds an optimum another misses. The sources differ in cost: Gauss-Newton reaches the optimum with half the evaluations of the cheapest "sr1" run, 638 against 1275 evaluations per optimum, the reverse of the ordering at nine parameters.

Two differences are structural rather than statistical.

For "sr1" an identity seed beats a Gauss-Newton seed, 79 starts against 65. The Gauss-Newton seed carries the curvature of a point the run is about to leave into the first steps of an approximation that is rebuilt anyway, and SR1 has no damping to discard it.

After the forward handover the model tests no longer fire, and 197 of 200 runs end on stagnation, at the values "gn" reaches alone. After the reversed handover 140 runs end on fvalue, because the Gauss-Newton model tests apply again. Neither changes which optimum is found: matched start by start, "gn" and the forward handover differ by  $3 \times 10^{-6}$ . By the time a soft stop fires the basin is chosen, and the handover only changes how the run ends inside it.

## 7 Discussion

"gn" and the two fallback runs reach the published optimum from the same number of starts. A fallback changes only how a run ends, moving stops from the model tests to **gradient**, and an "sr1" fallback moves more of them than a "bfgs" one.

On Boehm with the prior, "sr1" seeded from the identity reaches the best optimum from several times as many starts as "gn", 34 against 6 of 100, at a tenth of the evaluations per optimum.

**parscale**, the diagonal preconditioner, is off by default and in this vignette, and the quasi-Newton seed is not scaled either. On a logarithmic parameter scale, the normal case in dMod2, neither has anything to correct, and the ill-conditioning of sloppy models lies along rotated directions [13] that no diagonal scaling reaches. Scaling the seed is harmful: it takes one curvature value from the first accepted pair of a run and applies it to every direction, which a spectrum spread over decades does not support.

The ordering of the sources depends on the problem. On Bachmann Gauss-Newton reaches the optimum at half the cost of the cheapest "sr1" run; on Boehm "sr1" is cheapest, by an order of magnitude with the prior and by a factor of two without it. This is the direction theory predicts: where the residuals at the optimum are small,  $J^T J$  is close to the true Hessian, and a source that infers curvature from steps costs extra evaluations without improving the model. Where they are large, so is the omitted term, the Gauss-Newton model stops improving early, and its tests fire far from the minimum.

Three findings carry over between the settings. No source reaches an optimum another misses; they differ in cost. A handover does not change the basin a run ends in, since the basin is chosen before a soft stop fires. A quasi-Newton phase needs of the order of  $n_\theta$  accepted steps before it carries curvature everywhere, so its cost per optimum grows with the parameter count.

The forward handover changes only where inside a basin a run stops; the reversed one restores a valid stopping test.

## 8 Reproducing the numbers

The nine-parameter fits are stored in `dev/optimisation/results.rds` and reused, so this document renders in seconds and reproduces what it shows; `dev/render-optimisation.R --refit` recomputes them.

The two larger comparisons are too expensive for a render and are stored beside the document, one file each:

- `boehm_noprior.rds` comes from `bench_hessianSource.R` under `inst/benchmarks` with the environment variable `BOEHM_NOPRIOR` set, and takes minutes on one machine.
- `bachmann.rds` comes from `bachmann_hessianSource.R` under `dev/optimisation`, run first with `BACHMANN_SUBMIT` and then with `BACHMANN_COLLECT`. It needs a SLURM cluster: five job arrays of 50 tasks, four fits each.

Without `BOEHM_NOPRIOR`, `bench_hessianSource.R` runs the nine-parameter comparison of the earlier sections over the whole catalogue rather than the subset shown here, with the per-start `stopReason` tables.

## References

- [1] D. Kaschek, W. Mader, M. Fehling-Kaschek, M. Rosenblatt, and J. Timmer, “dMod: Efficient parameter estimation and uncertainty analysis for dynamic models,” *Journal of Statistical Software*, vol. 88, no. 10, pp. 1–32, 2019, doi: 10.18637/jss.v088.i10.
- [2] M. E. Boehm, L. Adlung, M. Schilling, S. Roth, U. Klingmüller, and W. D. Lehmann, “Identification of isoform-specific dynamics in phosphorylation-dependent STAT5 dimerization by quantitative mass spectrometry and mathematical modeling,” *Journal of Proteome Research*, vol. 13, no. 12, pp. 5685–5694, 2014, doi: 10.1021/pr5006923.
- [3] J. J. Moré and D. C. Sorensen, “Computing a trust region step,” *SIAM Journal on Scientific and Statistical Computing*, vol. 4, no. 3, pp. 553–572, 1983, doi: 10.1137/0904038.
- [4] T. F. Coleman and Y. Li, “An interior trust region approach for nonlinear minimization subject to bounds,” *SIAM Journal on Optimization*, vol. 6, no. 2, pp. 418–445, 1996, doi: 10.1137/0806023.
- [5] H. Zhang and W. W. Hager, “A nonmonotone line search technique and its application to unconstrained optimization,” *SIAM Journal on Optimization*, vol. 14, no. 4, pp. 1043–1056, 2004, doi: 10.1137/S1052623403428208.
- [6] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. Springer, 2006. doi: 10.1007/978-0-387-40065-5.
- [7] M. J. D. Powell, “A fast algorithm for nonlinearly constrained optimization calculations,” *Lecture Notes in Mathematics*, vol. 630, pp. 144–157, 1978, doi: 10.1007/BFb0067703.
- [8] D.-H. Li and M. Fukushima, “A modified BFGS method and its global convergence in nonconvex minimization,” *Journal of Computational and Applied Mathematics*, vol. 129, no. 1–2, pp. 15–35, 2001, doi: 10.1016/S0377-0427(00)00540-9.
- [9] F. Fröhlich and P. K. Sorger, “Fides: Reliable trust-region optimization for parameter estimation of ODE models,” *PLOS Computational Biology*, vol. 18, no. 7, p. e1010322, 2022, doi: 10.1371/journal.pcbi.1010322.
- [10] D. F. Shanno and K.-H. Phua, “Matrix conditioning and nonlinear optimization,” *Mathematical Programming*, vol. 14, no. 1, pp. 149–160, 1978, doi: 10.1007/BF01588962.
- [11] H. Hass, C. Loos, E. Raimúndez-Álvarez, J. Timmer, J. Hasenauer, and C. Kreutz, “Benchmark problems for dynamic modeling of intracellular processes,” *Bioinformatics*, vol. 35, no. 17, pp. 3073–3082, 2019, doi: 10.1093/bioinformatics/btz020.
- [12] J. Bachmann *et al.*, “Division of labor by dual feedback regulators controls JAK2/STAT5 signaling over broad ligand range,” *Molecular Systems Biology*, vol. 7, no. 1, p. 516, 2011, doi: 10.1038/msb.2011.50.
- [13] R. N. Gutenkunst, J. J. Waterfall, F. P. Casey, K. S. Brown, C. R. Myers, and J. P. Sethna, “Universally sloppy parameter sensitivities in systems biology models,” *PLOS Computational Biology*, vol. 3, no. 10, p. e189, 2007, doi: 10.1371/journal.pcbi.0030189.